# Clase 3: Redes Neuronales y Manifold Learning: Entrenamiento, Optimización y Generalización

Maldición de la dimensionalidad e  Hipótesis de manifold
Qué significa entrenar una red, Gradient Descent,  Learning Rate, Momentum y optimizadores adaptativos.
Overfitting y Regularización

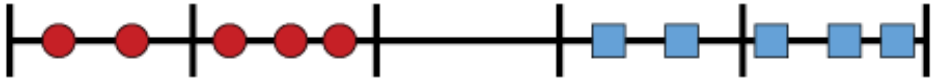
# La Maldición de la Dimensionalidad

Si bien una Red Neuronal es un aproximador universal, en la práctica existen grandes dificultades asociadas a: i) la cantidad y variedad de datos y ii) el entrenamiento.

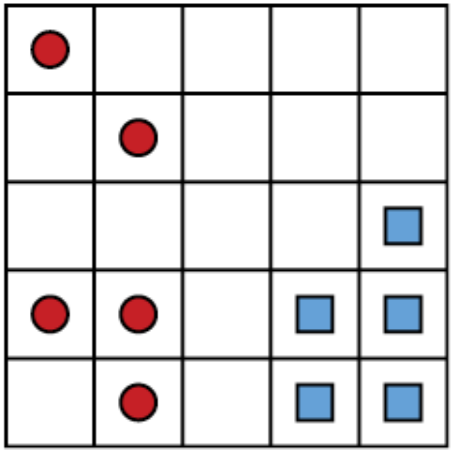
La “Maldición de la dimensionalidad” (Bellman 1961) surge del crecimiento exponencial en la cantidad de datos necesarios para representar el espacio de muestreo, al aumentar el número de inputs

# Un ejemplo tipo KNN: Azul vs Rojo

1 Dimension



2 Dimensiones



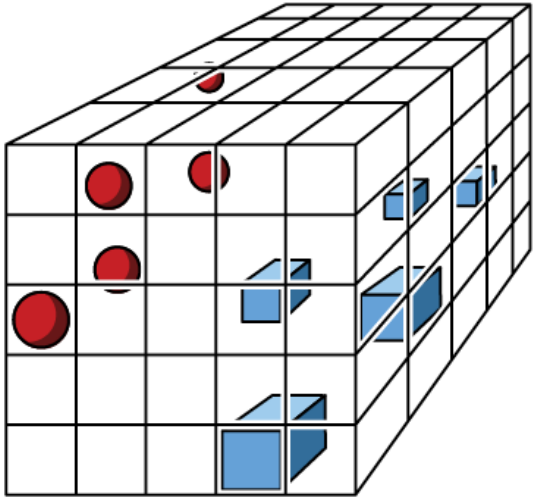
¿Cuantas casillas son “inclasificables?

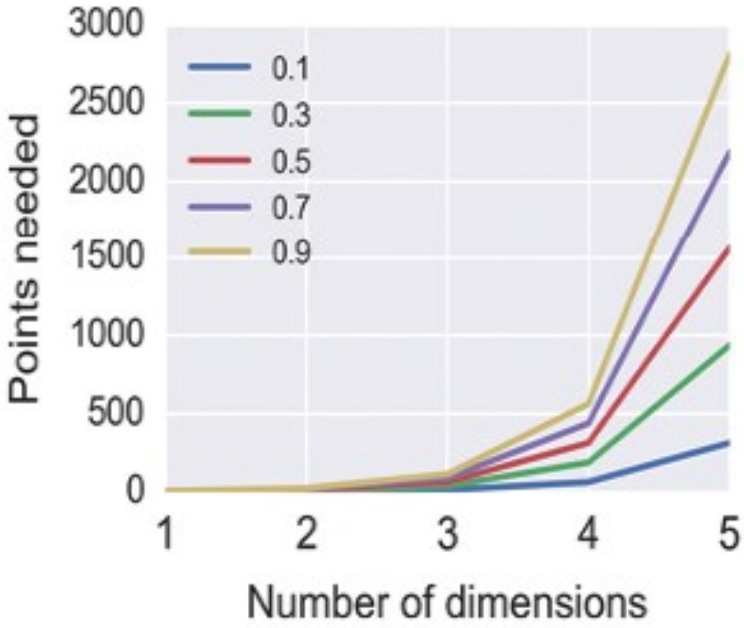
¿Cuántos datos más necesito para completarlo?

¿Cuán fácil es encontrar la frontera de bayes?

# Más dimensiones..

Al aumentar las dimensiones la cantidad de “puntos” (datos) necesarios para cubrir todo el espacio aumenta exponencialmente





# Algunos números en muchas dimensiones

La distancia de 2 puntos al azar en un cuadrado (l=1) es 0.52
La distancia de 2 puntos al azar en un cubo (l=1) es 0.66
En 1 millon de dimensiones (l=1) la distancia es casi 1.000.000

=> en muchas dimensiones el espacio está casi vacío (sparse).

=> nuevas “instancias” están lejos de las utilizadas para entrenar lo que lleva a bajo rendimiento

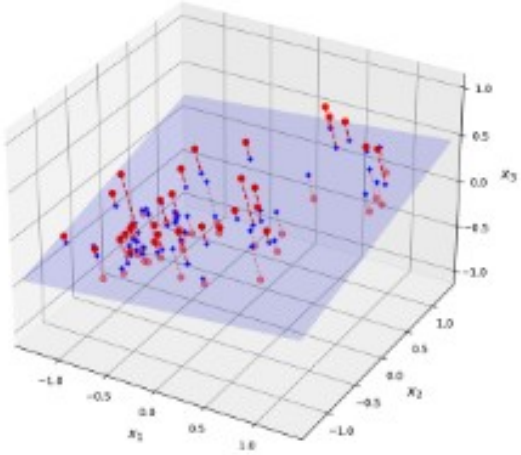
¿Soluciones?

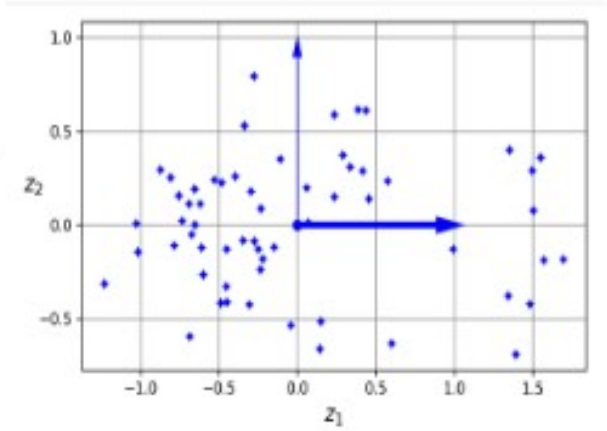
Note que para ‘sólo’ 100 dimensiones se necesitan más instancias de entrenamiento que átomos en el universo si queremos que entre 2 instancias no haya (en promedio) màs de 0.1 de distancia

# Proyecciones y “Manifold” learning

En la mayoría de los problemas reales, las instancias no se distribuyen uniformemente en el espacio sino que se agrupan en ciertos “sentidos” (poseen features altamente correlacionadas) =>existe una dimensón menor que los representa

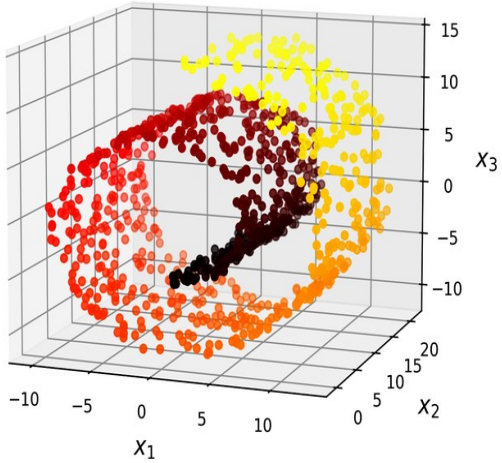
En 3D hay mucho espacio pero todos los puntos estan cerca del plano => 2D

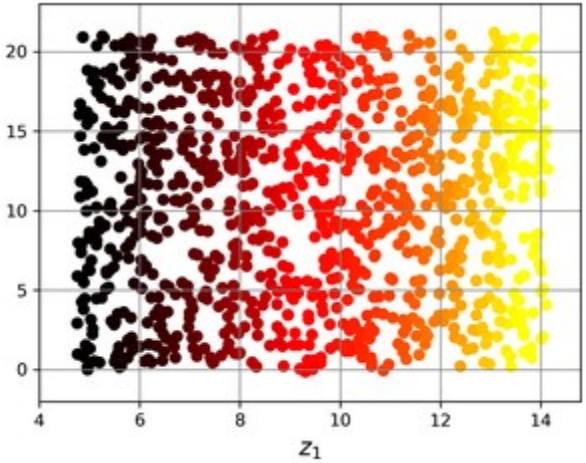




# Un “Manifold”

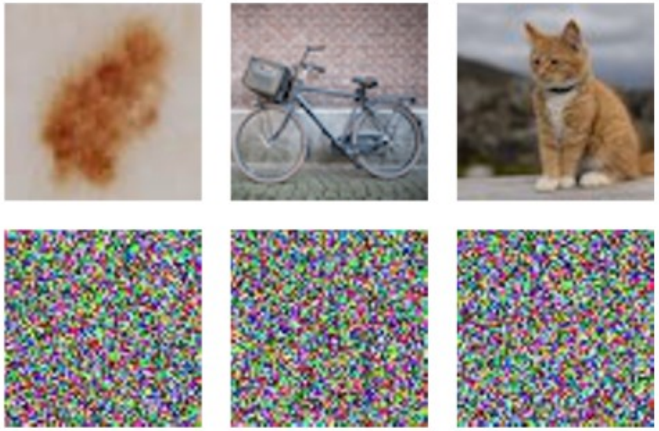
More generally, a d-dimensional manifold is a part of an n-dimensional space (where d < n) that locally resembles a d-dimensional hyperplane.

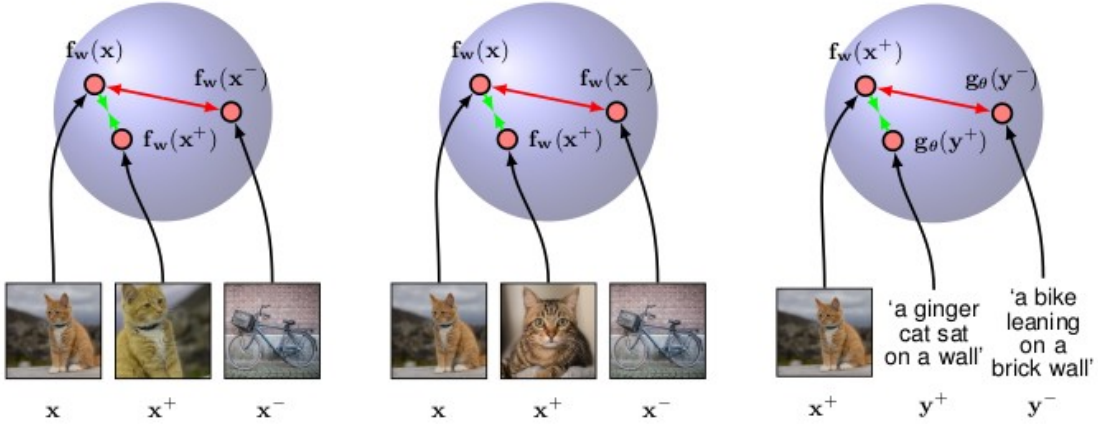



Los algoritmos de reducciòn de dimensionalidad se basan en buscar el manifold adecuado (asumiendo que existe). Muchos ejemplos “reales” parecen acompañar esta hipótesis

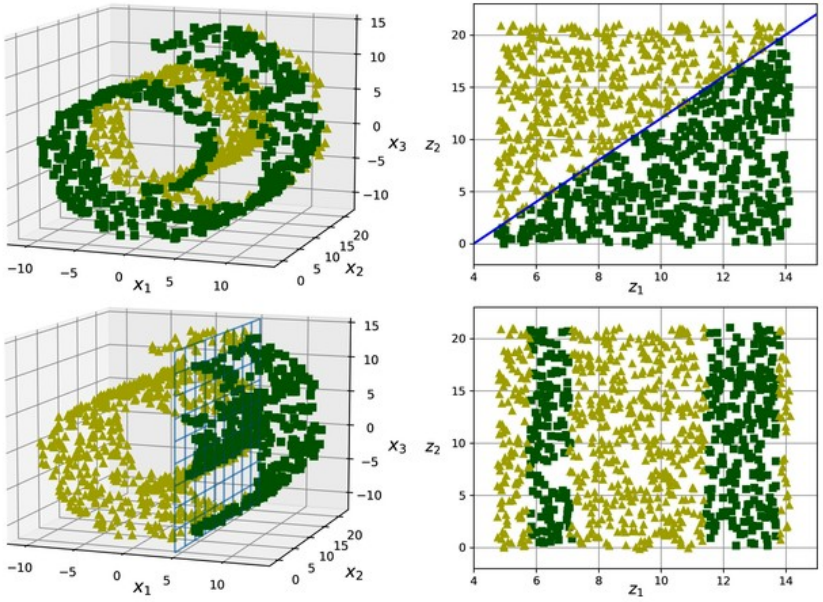
# Manifold en imágenes





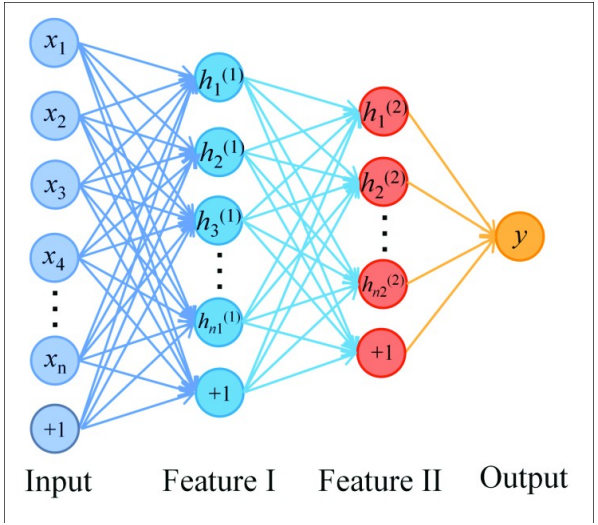
# La segunda hipótesis del Manifold

La 2da hipótesis del manifold learning sostiene que el problema a resolver (i.e clasificar las instancias) posee una solución màs simple en el manifold de dimensión reducida.



# Manifold => Redes neuronales

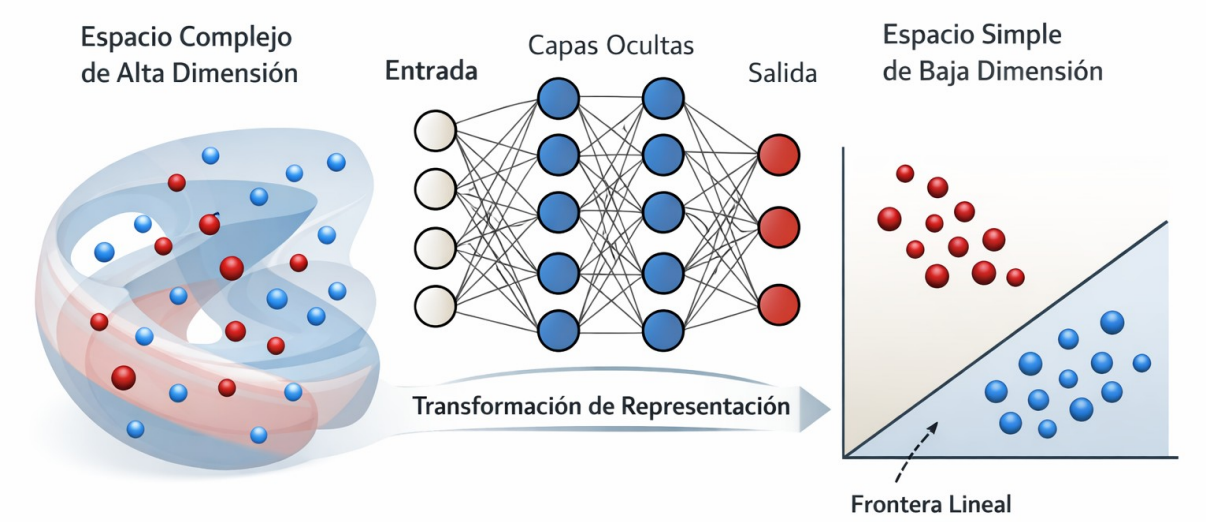
Las redes neuronales pueden considerarse métodos implícitos de manifold learning ya que las capas ocultas de una red neuronal transforman datos de alta dimensión en representaciones de menor dimensión



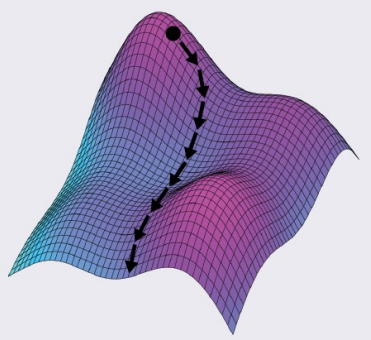

cada capa toma una representación de los datos y la transforma en otra más útil para la tarea.
Rol de las capas ocultas: reordenan, combinan y deforman el espacio de entrada para hacer el problema más simple.

- Una red neuronal no solo “predice”, sino que transforma los datos paso a paso. La red aprende (implícitamente) una transformación que lleva los datos desde un espacio complejo de alta dimensión a una representación donde  las clases están más separadas la estructura es más simple

- Interpretación: la red busca una nueva representación del dato donde una frontera simple (ej: lineal) sea suficiente.




Conclusión clave: Entrenar una red neuronal es aprender una secuencia de transformaciones que convierten un problema complejo en uno más simple en un nuevo espacio de representación.



# y el Entrenamiento.....

Conceptualmente el entrenamiento consiste en “mostrarle” a la red ejemplos de los cuales sabemos la respuesta, y de manera iterativa modificar los paràmetros (w,b) para que el error disminuya

Una vez entrenada, esperamos la red pueda resolver problemas por fuera del conjunto de entrenamiento

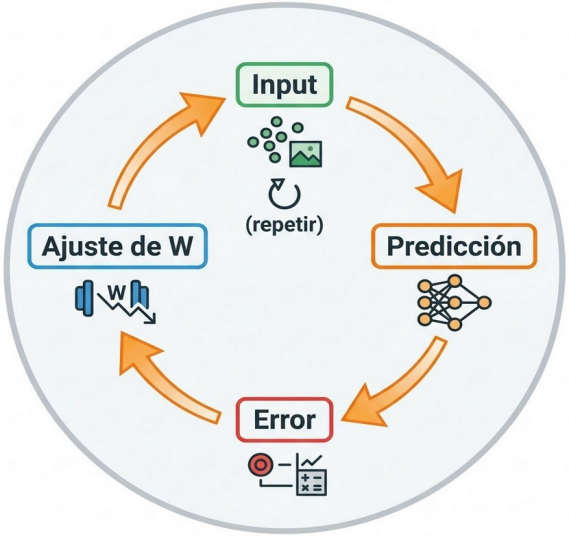
# ¿Qué significa entrenar una red?

=> es ajustar sus parámetros (pesos $W$ y biases $b$) para que sus predicciones se acerquen lo más posible a las respuestas correctas.

Ciclo de entrenamiento: Input → Predicción → Error → Ajuste de W → (repetir)

Objetivo: encontrar parámetros que minimicen el error en los datos de entrenamiento y generalicen a nuevos datos.

Conclusión clave: Entrenar una red es un problema de optimización: encontrar los parámetros que hacen que la red cometa el menor error posible.

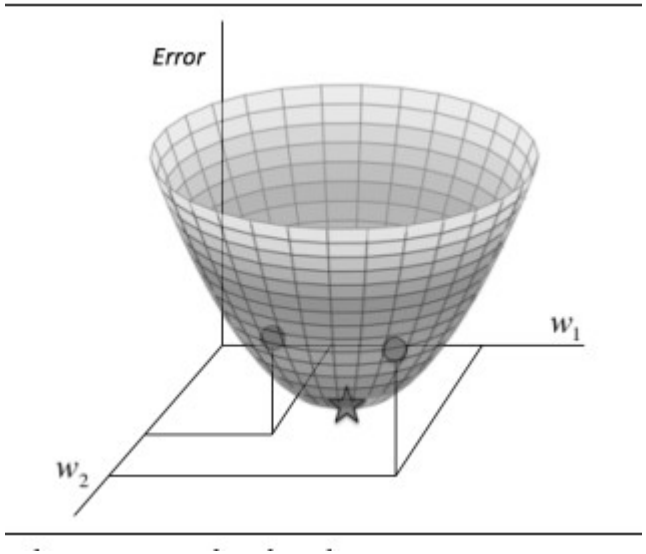


# Gradient Descent

Supongamos un problema con 2 inputs y 2 neuronas (sin bias), tengo solo 2 parámetros ¿cómo se verìa un gráfico 3D con eje-xy w1 y w2 yel error en el eje z?

La pendiente (gradiente) se hace cada vez más chica a media que me acerco al fondo del pozo

¿Podemos diseñar una estrategia que desde un punto al azar en la superficie nos lleve al mínimo?



# Gradient Descent

1-calculamos el gradiente (vector) en la posición inicial
2- damos un “paso” en esta dirección
3-calculamos el gradiente en la “nueva” posiciòn (màs cercana al mínimo)
4-esta serie de pasos conocida como “gradient descent” eventualmente nos lleva al mínimo

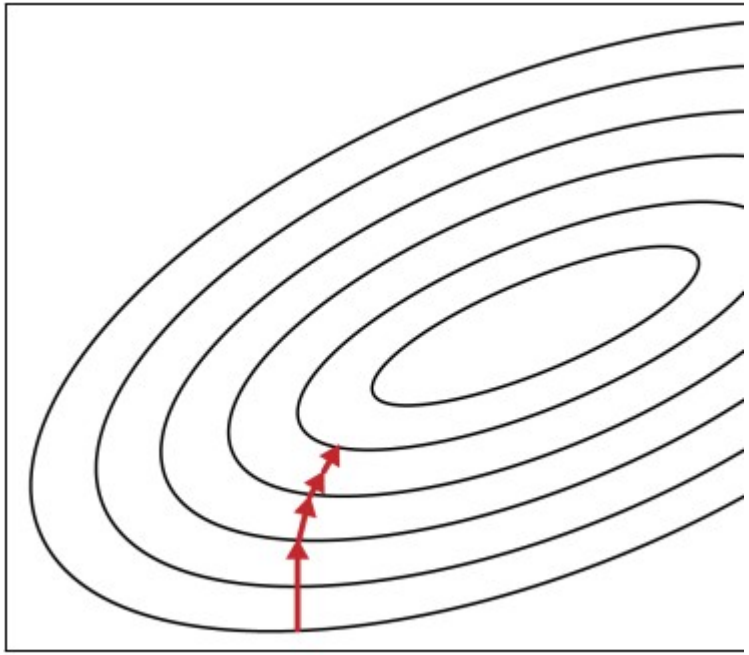


*Figure 4-3. Visualizing the error surface as a set of contours*

# El “Learning Rate”

La pregunta es:
¿de qué tamaño dar el paso?

Si el paso es muy pequeño, tarda mucho en llegar al mínimo

Si el paso es muy grande, me puedo pasar de largo

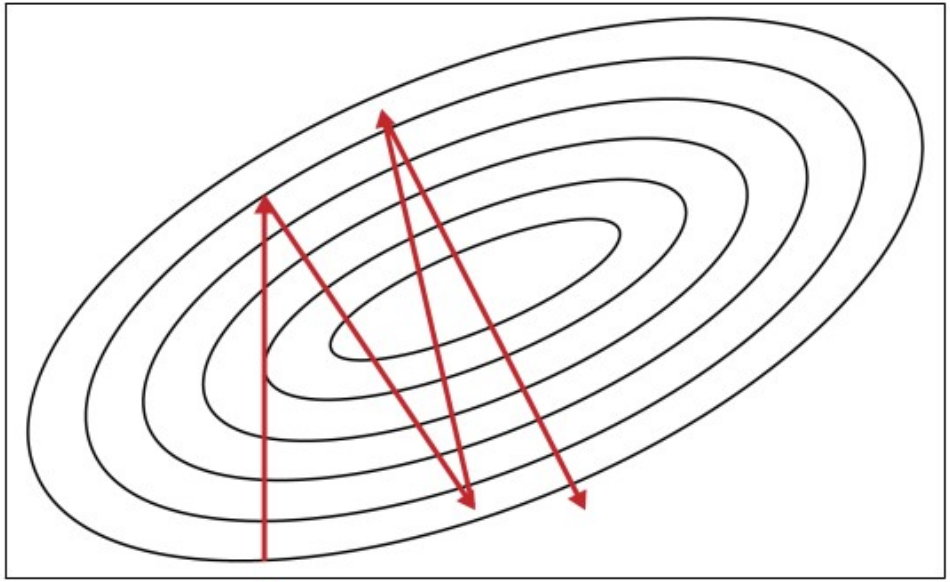


*Figure 4-4. Convergence is difficult when our learning rate is too large*

# Learning Rate(η)

- Trade-off: velocidad vs estabilidad
- Comportamiento típico:
  - al inicio: pasos más grandes ayudan
  - cerca del mínimo: conviene pasos más pequeños

- El learning rate determina cuán rápido y cuán estable aprende la red; elegirlo bien es crítico para que el entrenamiento converja.

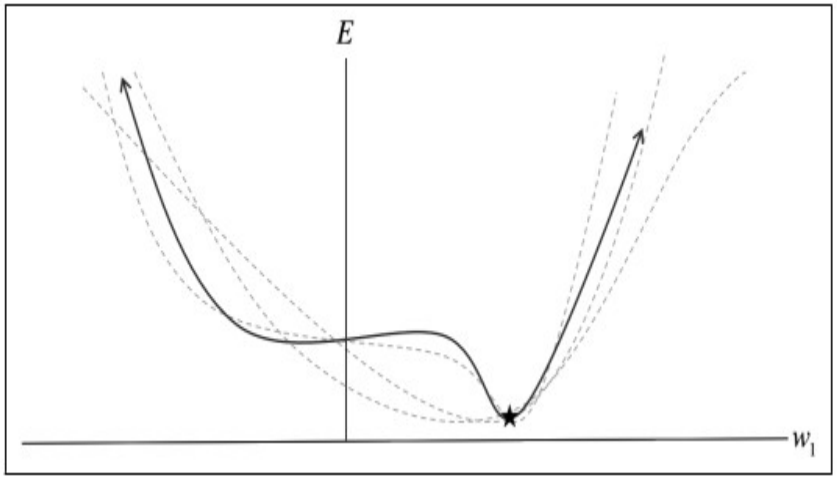
# ¿cuál es la relación del gradiente con el input?

El gradiente es “diferente” para cada input ¿entonces?

Batch GD: uso el promedio del gradiente de todos los inputs

Stochastic GD: uso los inputs de a uno en uno al azar

Mini-Batch GD: uso inputs en pequeños grupos (batches)

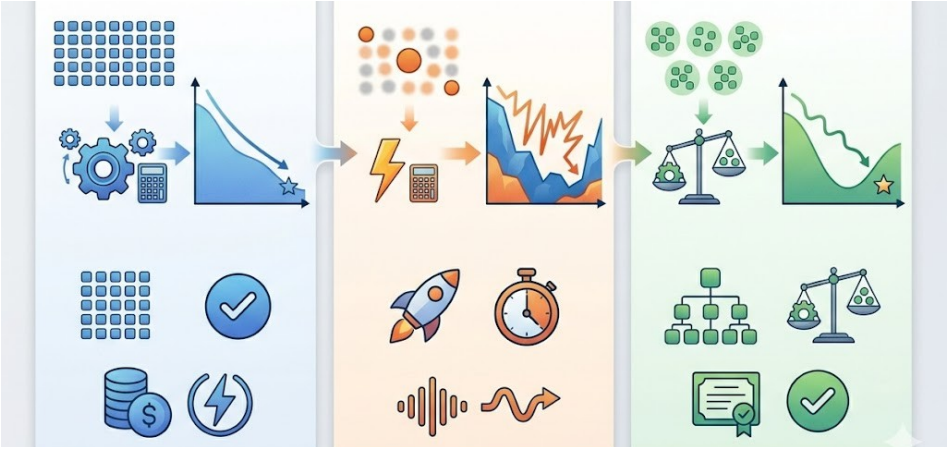


# Batch / Mini-batch / SGD

- Batch Gradient Descent:

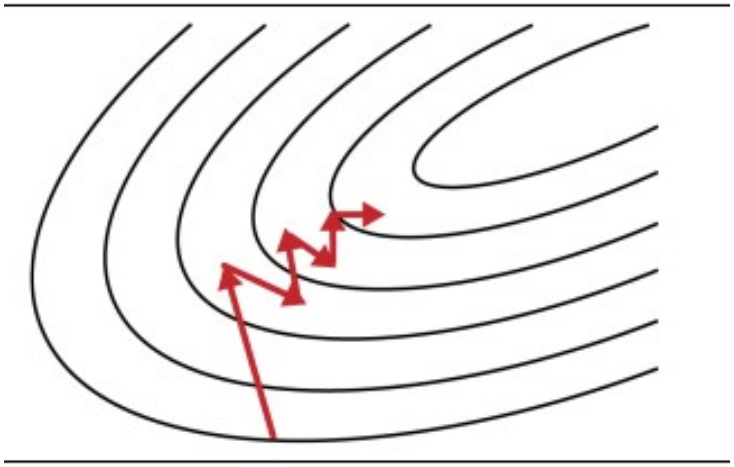
  usa todo el dataset para calcular el gradiente

  - actualización estable y precisa
  - pero computacionalmente costosa
- Stochastic Gradient Descent (SGD):
  - usa un solo ejemplo por actualización
  - muy rápido
  - pero ruidoso (alta variabilidad en los pasos)
- Mini-batch Gradient Descent:
  - usa pequeños grupos de datos (batches)
  - balance entre estabilidad y velocidad
  - es el método más usado en la práctica



# El uso de Momento

El gradiente es como una velocidad, pero si la hipersuperficie de error (HSE) es muy rugosa, este puede oscilar o moverse violentamente en una u otra dirección



Usar el “momento” permite tener en cuenta los gradientes de los pasos previos, dandole inercia al movimiento

$$v_t = \beta v_{t-1} + \eta \nabla L(w_t)$$

# Optimizadores Adaptativos

Ajusta automáticamente el LR para cada parámetro del modelo durante el entrenamiento permitiendo que cada peso aprenda a un ritmo distinto.

**Acelera la convergencia**, **Evita quedarse atascado en mínimos locales** y maneja bien **diferentes escalas de parámetros**

**Ejemplos:**

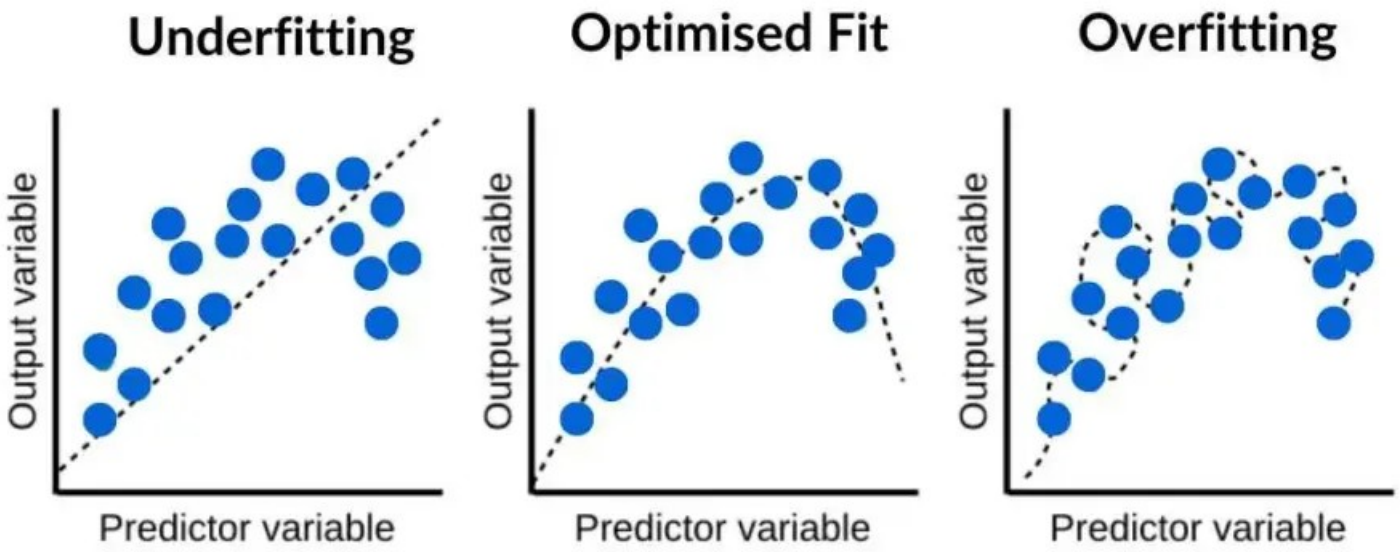
1. **Adagrad:** Disminuye la tasa de aprendizaje para parámetros que han tenido grandes actualizaciones.
2. **RMSprop:** Promedia los gradientes al cuadrado en una ventana móvil, ajustando la tasa de aprendizaje de cada peso según la variabilidad de los gradientes.
3. **Adam (Adaptive Moment Estimation):** Combina **momentum** y **RMSprop**.

# ENTRENAMIENTO DE REDES NEURONALES: OPTIMIZACIÓN EN UN ESPACIO NO CONVEXO

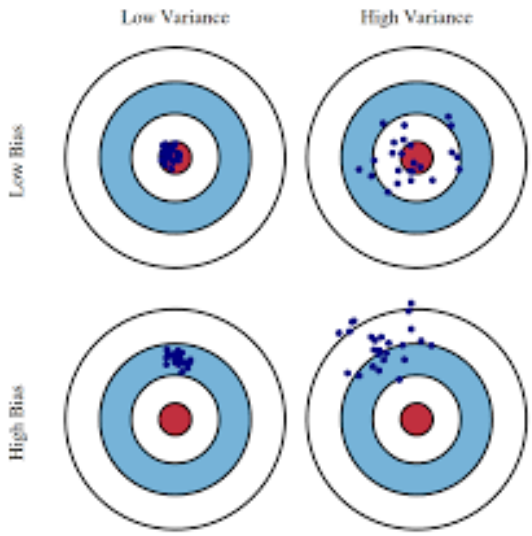
# Regularización

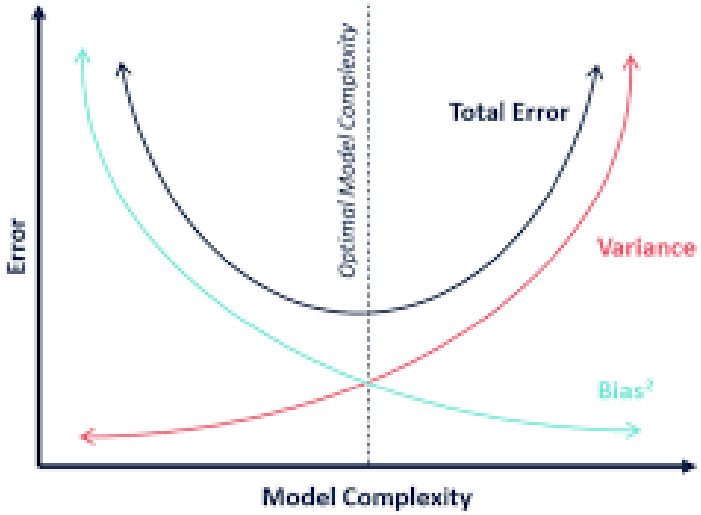
Son las técnicas diseñadas para prevenir el sobreajuste (overfitting) en los modelos

# Under y Overfitting



# Bias vs Variance trade-off

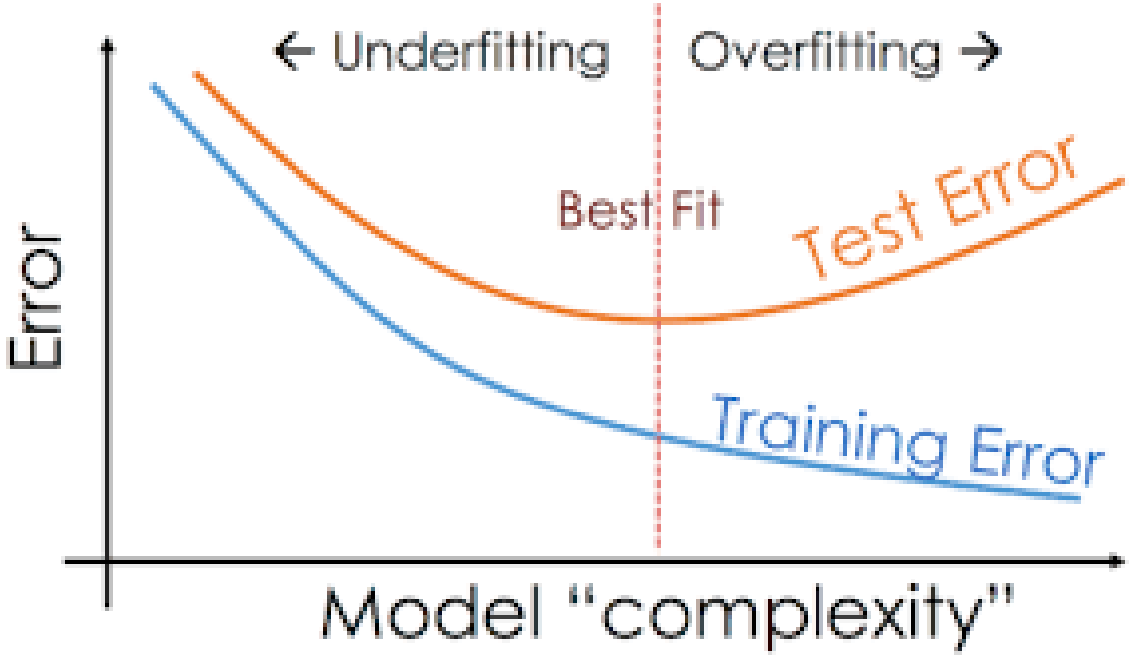



¿varianza de que?

En DNN ¿Dónde está el sesgo?, ¿Dónde la varianza? ¿ Cuál hay que corregir?

# ¿cómo lo observó?



# Overfitting

- ocurre cuando el modelo aprende demasiado bien los datos de entrenamiento, pero falla al generalizar a nuevos datos.
- El modelo no solo aprende la señal, sino también el ruido específico del dataset y/o aprende de menmoria.
- Causa: modelo demasiado flexible (muchos parámetros) pocos datos o datos ruidosos
- Esto posee Relación directa con bias-variance tardeoff: overfitting → alta varianza
-
- Objetivo: lo importante es entender que un buen modelo no es el que mejor ajusta el entrenamiento, sino el que mejor generaliza a datos nuevos.

| TRAIN SET | VALIDATION SET | TEST SET |
|---|---|---|
| Datos de Entrenamiento | Datos de Validación | Datos de Testeo |
| El modelo aprende aquí | *Ajuste de Hiperparámetros* | Evaluación Final |

# Train vs Validation vs Test

- Train (entrenamiento) Datos usados para ajustar los parámetros del modelo → el modelo “aprende” aquí
- Validation (validación) Datos no vistos durante entrenamiento → se usan para ajustar hiperparámetros (LR, arquitectura, etc.)
- → permiten monitorear overfitting
- Test (evaluación final) Datos completamente no vistos → se usan solo al final para medir performance real
- Comportamiento típico:
  - Error Train "Baja" continuamente
  - Error Validation/Test: Baja al inicio (el modelo aprende) luego sube (overfitting)
- Conclusión clave: Un modelo es bueno si tiene bajo error en validation/test, no solo en train.

# Regularización L1 y L2

L2 agrega a la LF $wi^2$, de este modo cuantos más altos los pesos más grande la LF.
El resultado es que durante el entrenamiento el sistema busca que los wi sean pequeños y ma parejo entre sí, evitando que algunas neuronas que responden a inputs específicos tengan “literalmente” demasiado peso

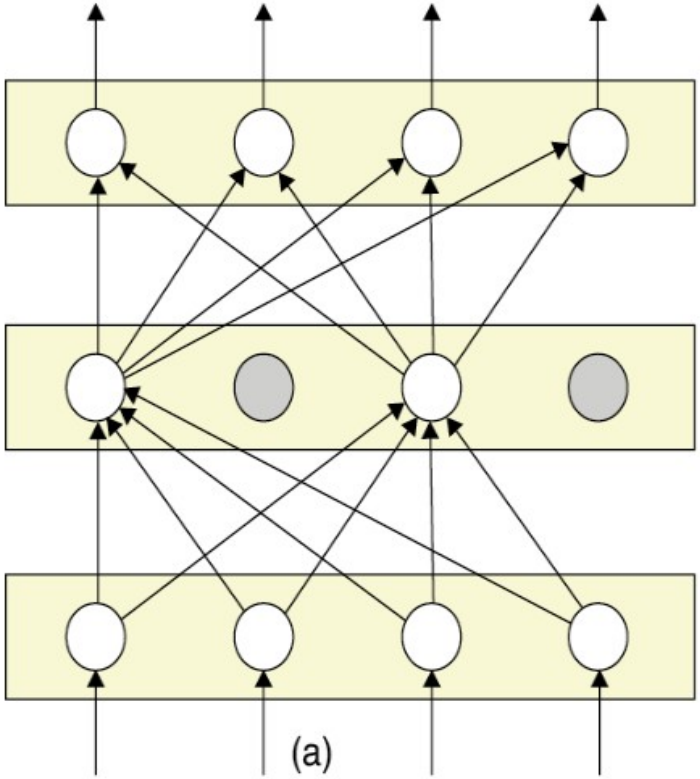
L1 se agrega un termino λw a cada wi de la red buscando que estos minimicen su valor. De este modo durante el entrenamiento algunos λw tienden a 0. Esto fuerza a las neuronas a usar “sòlo” los inputs más importantes y de este modo el modelo se hace más robusto.

# Dropout

Consiste en desactivar aleatoriamente un porcentaje de neuronas (junto con sus conexiones) durante el entrenamiento, en cada iteración. Esto evita que el modelo dependa demasiado de neuronas específicas, obligándolo a aprender representaciones más robustas y generalizables.

En la práctica, se define una tasa de dropout ($p$), como por ejemplo $p$=0.5, lo que significa que el 50% de las neuronas se apagan en cada forward pass.

Durante la fase de inferencia (evaluación), dropout se desactiva y todas las neuronas contribuyen al resultado.



# Batch Normalization

En cada “mini-batch” se normaliza las activaciones de cada capa, para que tengan media cercana a 0 y varianza cercana a 1.

Pero además cada neurona tiene dos parámetros adicionales (entrenables) gamma y beta que escala y desplaza las activaciones (en un modelo lineal), permitiendo que el modelo ajuste la amplitud de la distribución.

$$y_i = \left(\frac{x_i - \mu}{\sigma}\right)\gamma + \beta$$

# Resumiendo:

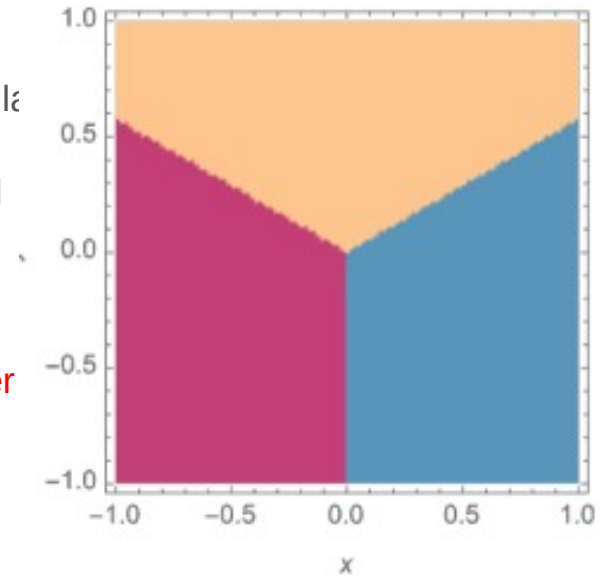
- En alta dimensión, los datos son dispersos → necesitamos estructura (manifold)
- Las redes neuronales aprenden representaciones que simplifican el problema
- Entrenar = ajustar parámetros para minimizar el error
- Gradient Descent permite optimizar, controlado por el learning rate
- Mini-batch introduce un balance entre eficiencia y estabilidad
- Optimizadores mejoran la convergencia (momentum, Adam, etc.)
- El problema es no convexo → no hay solución única
- Riesgo de overfitting → el modelo puede memorizar
- La regularización permite mejorar la generalización
- **Idea final: Entrenar una red es aprender una representación útil de los datos que generalice bien, no solo que ajuste el entrenamiento.**

# A ver si entendimos

- ¿Por qué la “maldición de la dimensionalidad” hace difícil aprender a partir de datos en alta dimensión?
- ¿Qué significa que los datos vivan en un manifold y por qué eso ayuda al aprendizaje?
- ¿Qué está haciendo una red neuronal cuando transforma los datos capa a capa?
- ¿Por qué Gradient Descent no garantiza encontrar el mínimo global en redes neuronales?
- ¿Cómo se detecta el overfitting y qué técnicas se pueden usar para reducirlo?
- ¿Qué hace conceptualmente la regularización?

# Y ahora el TP3

Objetivo: En este trabajo pràctico extenderemos el aprendizaje de los fundamentos de redes neuronales, para lo cuál adaptaremos nuestra pequeña red para un problema de clasificaciòn, para lo cuál utilizaremos la funciòn softmax. Tambièn aprovecharemos la mayor complejidad del problema para analizar los efectos de la cantidad (y calidad) de datos del aprendizaje.




3.1 una Clasificación bidimensional en 3 categorìas

Para ir calentando motores cree un código en python que permita obtener una grilla de valores (x,y) en el rango (-1,1) con la clasificación correcta (R,Az, Am) de acuerdo a lo que se muestra en la figura.

Nota: lo ideal para lo que viene es que su codigo permita ajustar el espaciamiento entre los valores

Nota 2: la intersección de la la frontera entre R/Am y Az/Am debe estar en 0.5 pero no tiene que srer exacto. Mantenga la triple frontera en el (0,0)

Ejercicio 3.1)

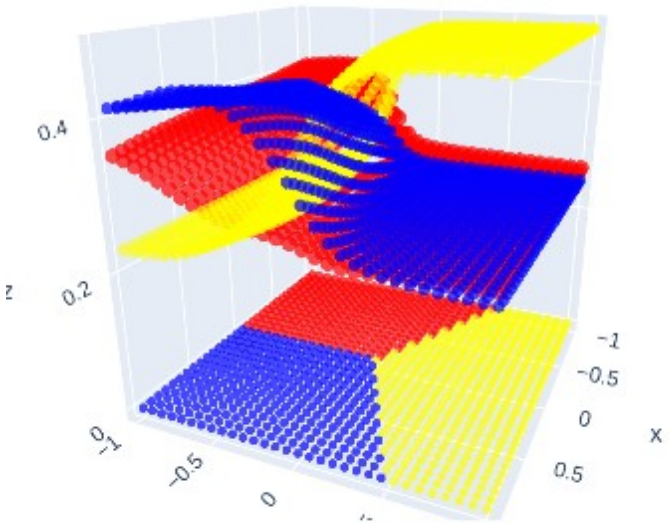
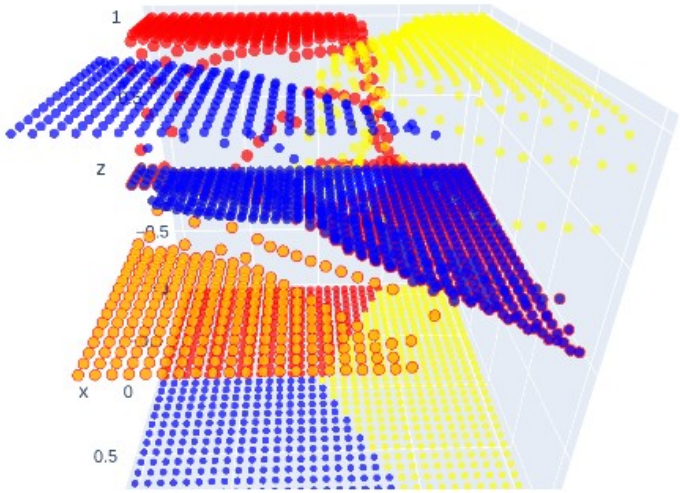
a) Escriba un codigo que permita entrenar y evaluar la pequeña red neuronal para resolver el problema de clasificaciòn. Al igual que en el caso anterior analicè el efecto del LR, #E y arquitectura de la red. En este caso, como el input es bidimensional (x,y) en el intervalo (-1,1), el nùmero de datos de entrada crece como n2, (n = 2/s, s=step entre puntos) lo que permite generar fàcilmente mayor cantidad de los mismos. Analicé el efecto del mismo en relación al tiempo de entrenamiento y la capacidad de predicciòn de la red.

b) Para analizar los resultados ademas del valor de la LF final, analicè la “Accuracy” (% de predicciones correctas) y la matriz de confusiòn.

c) Para analizar con mayor profundidad la performance de la red, puede realizar un análisis gràfico. Como el que se muestra en el siguiente ejemplo (en donde sobre un plano con cada uno de los colores en z=o, se representa en el eje z la probabilidad de cada color)

Ejercicio 3.2) Mini Bacth-training.

Vamos a aprovechar nuestra pequeña red neuronal para profundizar sobre las técnicas de entrenamiento. Modifique su algoritmo de entrenamiento para que en vez de actualizar los paràmetros (w,b) cada època (o sea luego de utilizar todos los paràmetros), lo haga luego de realizar el fwd y backprop con una pequeña fraccion de los datos (mini batch). Recuerde que en el mini batch si hay 100 datos y el batch-size es 20, los paràmetros (w,b) se actualizan de a pasadas de a 20, y recien cuando se completaron las 5 pasadas de 20 (o sea los 100 datos) se completa un epoca. Reentrene la red con este algoritmo y compare la velocidad de entrenamiento y el resultado final.

Ejercicio 3.3) Modifique su pequeña red neuronal para que en lugar de la sigmoidea utilice ReLu. Entrenale y compare a) la velocidad y capacidad de entrenamiento b) el resultado, c) el efecto de diferentes arquitecturas con la red previa utilizando la funciòn sigmoidea.

# imagenes...